`std::constant_wrapper`

Document #:	P2781R4
Date:	2023-06-10
Project:	Programming Language C++
Audience:	LEWG-I LEWG

Reply-to:	Matthias Kretz <m.kretz@gsi.de> Zach Laine <whatwasthataddress@gmail.com>

Contents

- 1 Changelog
  - 1.1 Changes since R0
  - 1.2 Changes since R1
  - 1.3 Changes since R2
  - 1.4 Changes since R3
- 2 Relationship to previous work
- 3 The ergonomics of `std::integral_constant<int>` are bad
  - 3.1 Replacing the uses of `std::integral_constant` is not enough
  - 3.2 The difference in template parameters to `std::constant_wrapper` and `std::cw`
- 4 Making `constant_wrapper` more useful
- 5 What about strings?
- 6 An example using `operator()`
- 7 What about the mutating operators?
- 8 What about `operator->`?
- 9 Convertibility to and from `std::integral_constant`
- 10 Design
  - 10.1 Add `constant_wrapper`
  - 10.2 Add a feature macro
- 11 Implementation experience
- 12 Wording

§ 1 Changelog

§ 1.1 Changes since R0

- Remove discussion of literals for `std::integral_constant` and `std::constexpr_v`, based on LEWG feedback.
- Change the title to reflect the loss of the literals.

— Add an explicit example involving `std::constexpr_v::operator()`, as requested by LEWG.

— Add concept `std::constexpr_value`, as suggested by LEWG.

— Wording.

1.2 Changes since R1

— Remove the `constexpr_value` concept.

— Use an `auto` NTTP parameter for `constexpr_v`, and default its type template parameter.

— Add the option for adding the mutating overloadable operators.

— Add a note about `operator->`.

— Add remarks about interconvertibility with `std::integral_constant`.

— Reduce the number of naming options.

— Simplify the implementation.

1.3 Changes since R2

— Remove unnecessary uses of trailing return type.

— Remove use of `and`, `or` and `not` keywords.

— Correct the requirements in the exposition only *`constexpr-param`* concept.

— Add a note about the imperfect nature of ADL support given by `constexpr_v`'s `T` template parameter.

1.4 Changes since R3

— Fix IFNDR in unary operator overloads (including `operator()` and `operator[]`).

— Add mutating operators, like `operator++` and `operator-=`.

— Change the defaulted template parameter of the `constexpr_v` (formerly "`T`") to be exposition-only.

— `constexpr_v` -> `constant_wrapper`, and `c_` -> `cw`.

2 Relationship to previous work

This paper is co-authored by the authors of P2725R1 ("`std::integral_constant` Literals") and P2772R0 ("`std::integral_constant` literals do not suffice — `constexpr_t`?"). This paper supersedes both of those previous papers.

3 The ergonomics of `std::integral_constant<int>` are bad

`std::integral_constant<int>` is used in lots of places to communicate a constant integral value to a given interface. The length of its spelling makes it very verbose. Fortunately, we can do a lot better.

Before

```
// From P2630R1
auto const sir =
  std::strided_index_range{std::integral_constant<size_t, 0>{},
                           std::integral_constant<size_t, 10>
        {},
                           3};
auto y = submdspan(x, sir);
```

After

```
auto y = submdspan(x,
         std::strided_index_range{
    std::cw<0>, std::cw<10>, 3});
```

The “after” case above would require that `std::strided_index_range` be changed; that is not being proposed here. The point of the example is to show the relative convenience of `std::integral_constant` versus the proposed `std::constant_wrapper`.

3.1 Replacing the uses of `std::integral_constant` is not enough

Parameters passed to a `constexpr` function lose their `constexpr`-ness when used inside the function. Replacing `std::integral_constant` with `std::constant_wrapper` has the potential to improve a lot more uses of compile-time constants than just integrals; what about all the other `constexpr`-friendly C++ types?

Consider:

```
template<typename T>
struct my_complex
{
    T re, im;
};

inline constexpr short foo = 2;

template<typename T>
struct X
{
    void f(auto c)
    {
        // c is to be used as a constexpr value here
    }
};
```

We would like to be able to call `X::f()` with a value, and have that value keep its `constexpr`-ness. Let’s introduce a template “`constant_wrapper`” that holds a `constexpr` value that it is given as an non-type template parameter.

```
namespace std {
  template<auto X>
  struct constant_wrapper
  {
    using value_type = remove_cvref_t<decltype(X)>;
    using type = constant_wrapper;

    constexpr operator value_type() const { return X; }
    static constexpr value_type value = X;

    // The rest of the members are discussed below ....
  };
}
```

Now we can write this.

```
template<typename T>
void g(X<T> x)
{
    x.f(std::constant_wrapper<1>{});
    x.f(std::constant_wrapper<2uz>{});
    x.f(std::constant_wrapper<3.0>{});
    x.f(std::constant_wrapper<4.f>{});
    x.f(std::constant_wrapper<foo>{});
    x.f(std::constant_wrapper<my_complex(1.f, 1.f)>{});
}
```

Let’s now add a `constexpr` variable template with a shorter name, say `cw`.

```
namespace std {
  template<auto X>
  inline constexpr constant_wrapper<X> cw{};
}
```

And now we can write this.

```
template<typename T>
void g(X<T> x)
{
    x.f(std::cw<1>);
    x.f(std::cw<2uz>);
    x.f(std::cw<3.0>);
    x.f(std::cw<4.f>);
    x.f(std::cw<foo>);
    x.f(std::cw<my_complex(1.f, 1.f)>);
}
```

3.2 The difference in template parameters to `std::constant_wrapper` and `std::cw`

If you look at the wording below, you will see that `std::cw` takes an `auto` NTTP, whereas `std::constant_wrapper` takes an `auto` NTTP `X`, and an exposition-only parameter *`adl-type`* which is defaulted to `remove_cvref_t<decltype(X)>`. Why is this? As the *`adl-type`* name implies, ADL! Even though the type of `X` is deduced with or without *`adl-type`*, without it some natural uses of `constant_wrapper` cease to work. For instance:

```
auto f = std::cw<strlit("foo")>; // Using the strlit from later in this paper.
std::cout << f << "\n";
```

The stream insertion breaks without the *`adl-type`* parameter. *`adl-type`* is `strlit</*...*/>`, which pulls `strlit`'s `operator<<` into consideration during ADL. Note that this ADL support is imperfect. The use op `operator<<` above is due to the way the operator overload is declared:

```
friend std::ostream & operator<<(std::ostream & os, strlit l) { /* ...*/ }
```

If it is instead declared as a non-`friend`:

```
template<size_t N>
std::ostream & operator<<(std::ostream & os, strlit<N> l) { /* ...*/ }
```

… ADL's help doesn't suffice. The deduction of `N` is not possible from a type that isn't a `strlit<N>` itself (e.g. base class) even if it is implicitly convertible to `strlit<N>`.

4 Making `constant_wrapper` more useful

`constant_wrapper` is essentially a wrapper. It takes a value `X` of some structural type `value_type`, and represents `X` in such a way that we can continue to use `X` as a compile-time constant, regardless of context. As such, `constant_wrapper` should be implicitly convertible to `value_type`; this is already reflected in the design presented above. For the same reason, `constant_wrapper` should provide all the operations that the underlying type has. Though we cannot predict what named members the underlying type `value_type` has, we *can* guess at all the operator overloads it might have.

So, by adding conditionally-defined overloads for all the overloadable operators, we can make `constant_wrapper` as natural to use as many of the types it might wrap.

```
namespace std {
  template<auto X>
  struct constant_wrapper {
    using value_type = remove_cvref_t<decltype(X)>;
    using type = constant_wrapper;

    constexpr operator value_type() const { return X; }
    static constexpr value_type value = X;

    // unary -
```

```
    template<auto Y = X>
      constexpr constant_wrapper<-Y> operator-() const { return {}; }

    // binary + and -
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<U::value + V::value> operator+(U, V) { return {}; }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<U::value - V::value> operator-(U, V) { return {}; }

    // etc... (full listing later)
  };
}
```

These operators are defined in such a way that they behave just like the operations on underlying the `U` and `V` values would, including promotions and coercions. For example:

```
static_assert(std::is_same_v<
              decltype(std::cw<42> - std::cw<13u>),
              std::constant_wrapper<29u>>);
```

Each operation is only defined if the underlying operation on `X` is defined. Each operation additionally requires that the result of the underlying operation have a structural type.

All the overloadable operations are included, even the index and call operators. The rationale for this is that a user may want to make some sort of compile-time domain-specific embedded language using operator overloading, and having all but a couple of the operators specified would frustrate that effort. The only exception to this is `operator->` which must eventually return a pointer type, which is not very useful at compile time.

The only downside to adding `std::constant_wrapper::operator()` is that it would represent a break from the design of `std::integral_constant`, making it an imperfect drop-in replacement for that template. Nullary `std::constant_wrapper::operator()` with the same semantics as `std::integral_constant::operator()` is defined when `requires (!std::invocable<value_type>)` is `true`, so this incompatibility is truly a corner case.

The operators are designed to interoperate with other types and templates that have a constexpr static `value` member. This works with `std::constant_wrappers` of course, but also `std::integral_constants`, and user-provided types as well. For example:

```
struct my_type { constexpr static int value = 42; };

void foo()
{
    constexpr auto zero = my_type{} - std::cw<42>;  // Ok.
    // ...
}
```

Note that the addition of these operators is in line with the poll:

“Add a new robust integral constant type with all the numerical operators, as proposed in P2772R0, and use that for these literals instead of `std::integral_constant`”?

SF	F	N	A	SA
4	7	1	1	1

… taken in the 2023-01-17 Library Evolution telecon.

Note that the one SA said he would not be opposed if the word “integral” was stricken from the poll, and the design of `std::constant_wrapper` is not limited to integral types.

§ 5 What about strings?

As pointed out on the reflector, `std::cw<"foo">` does not work, because of language rules. However, it's pretty easy for users to add an NTTP-friendly string wrapper type, and then use that with `std::cw<>`.

```
template<size_t N>
struct strlit
{
    constexpr strlit(char const (&str)[N]) { std::copy_n(str, N, value); }

    template<size_t M>
    constexpr bool operator==(strlit<M> rhs) const
    {
        return std::ranges::equal(bytes_, rhs.bytes_);
    }

    friend std::ostream & operator<<(std::ostream & os, strlit l)
    {
        assert(!l.value[N - 1] && "value must be null-terminated");
        return os.write(l.value, N - 1);
    }

    char value[N];
};

int main()
{
    auto f = std::cw<strlit("foo")>;
    std::cout << f; // Prints "foo".
}
```

6 An example using `operator()`

The addition of non-arithmetic operators may seem academic at first. However, consider this `constexpr`-friendly parser combinator mini-library.

```
namespace parse {

    template<typename L, typename R>
    struct or_parser;

    template<size_t N>
    struct str_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return lit == str_;
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<str_parser, P>{*this, parser};
        }
        strlit<N> str_;
    };

    template<typename L, typename R>
    struct or_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return l_(lit) || r_(lit);
        }
        template<typename P>
        constexpr auto operator|(P parser) const
```

```
        {
            return or_parser<or_parser, P>{*this, parser};
        }
        L l_;
        R r_;
    };

}

int foo()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p = p1 | p2 | p3;

    constexpr bool matches_empty = p(strlit(""));
    static_assert(!matches_empty);
    constexpr bool matches_pos = p(strlit("pos"));
    static_assert(!matches_pos);
    constexpr bool matches_decr = p(strlit("decr"));
    static_assert(matches_decr);
}
```

(This relies on the `strlit` struct shown just previously.)

Say we wanted to use the templates in namespace `parser` along side other values, like `int`s and `float`s. We would want that not to break our `std::constant_wrapper` expressions. Having to work around the absence of `std::constant_wrapper::operator()` would require us to write a lot more code. Here is the equivalent of the function `foo()` above, but with all the variables wrapped using `std::cw`.

```
int bar()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p_ = std::cw<p1> | std::cw<p2> | std::cw<p3>;

    constexpr bool matches_empty_ = p_(std::cw<strlit("")>);
    static_assert(!matches_empty_);
    constexpr bool matches_pos_ = p_(std::cw<strlit("pos")>);
    static_assert(!matches_pos_);
    constexpr bool matches_decr_ = p_(std::cw<strlit("decr")>);
    static_assert(matches_decr_);
}
```

As you can see, everything works as it did before. The presence of `operator()` does not enable any new functionality, it just keeps code that happens to use it from breaking.

§ 7 What about the mutating operators?

It may seem at first that these operators are nonsensical, since all the operations on a `constant_wrapper` must be nonmutating.

However, some DSLs may wish to use these operations with atypical semantics.

```
struct weirdo
{
    constexpr int operator++() const { return 1; }
};
auto result = ++std::cw<weirdo{}>;
```

`result` is obviously `std::cw<1>` here, and no mutation occurred. You can imagine a more elaborate use case, say a library that is used to create expression templates. For example:

```
auto expr = std::cw<var0> += std::cw<var1>;
```

In this case, `var0` and `var1` would be some terminal types in the expression template library, and `operator+=` would return a `constexpr` expression tree, rather than mutating the left side of the `+=`.

These operators are now part of the proposal, based on this LEWG poll from Kona 2023:

"We should add mutating operations (i.e. `#define IF_LEWG_SAYS_SO 1` and `++` and `--`) to P2781R3"

SF	F	N	A	SA
2	6	5	2	0

8 What about `operator->`?

We're not proposing it, because of its very specific semantics – it must yield a pointer, or something that eventually does. That's not a very useful operation during constant evaluation.

9 Convertibility to and from `std::integral_constant`

During the LEWG reviews, some attendees suggested that inter-conversions between `std::integral_constant` and `std::constant_wrapper` would be useful. The important thing to remember is that we want deduction to occur when calling functions that take a `std::constant_wrapper`, including the `std::constant_wrapper` operator overloads. Conversions and deductions are at odds with one another, because deducing parameter types disables the conversion rules.

If you look at the operator overloads proposed here, you will see that they are deduction operations at their most essential. The types of the parameters do not matter, except that each conveys a value that is a core constant expression because it is embedded in the type system. The fact that a `std::constant_wrapper` conveys that value instead of a `std::integral_constant` is immaterial, and in fact the operators are written in such a way that they operate on either template (as long as at least one parameter is a specialization of `std::constant_wrapper`). Users can and should write their code using these kinds of values-as-types in a similar way. Relying on conversions is a less-useful way to get interoperability.

10 Design

10.1 Add `constant_wrapper`

```
namespace std {
  template<auto X,
           class adl-type = remove_cvref_t<decltype(X)>>     // exposition only
    struct constant_wrapper;

  template <class T>
    concept constexpr-param =                                // exposition only
      requires { typename constant_wrapper<T::value>; };
  template <class T>
    concept derived-from-constexpr =                         // exposition only
      derived_from<T, constant_wrapper<T::value>>;
  template <class T, class SelfT>
    concept lhs-constexpr-param =                            // exposition only
      constexpr-param<T> && (derived_from<T, SelfT> || !derived-from-constexpr<T>);

  template<auto X, class adl-type>
```

```
struct constant_wrapper {
  using value_type = adl-type;
  using type = constant_wrapper;

  constexpr operator value_type() const { return X; }
  static constexpr value_type value = X;

  template <constexpr-param U>
    constexpr constant_wrapper<(X = U::value)> operator=(U) const { return {}; }

  template<auto Y = X>
    constexpr constant_wrapper<+Y> operator+() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<-Y> operator-() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<~Y> operator~() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<!Y> operator!() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<&Y> operator&() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<*Y> operator*() const { return {}; }

  template<auto Y = X, class... Args>
     constexpr constant_wrapper<Y(Args::value...)> operator()(Args... args) const { return
      {}; }
  template<auto Y = X, class... Args>
     constexpr constant_wrapper<Y[Args::value...]> operator[](Args... args) const { return
      {}; }
  constexpr value_type operator()() const requires (!std::invocable<value_type>) { return X;
      }

  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value + V::value> operator+(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value - V::value> operator-(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value * V::value> operator*(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value / V::value> operator/(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value % V::value> operator%(U, V) { return {}; }

  template <lhs-constexpr-param<type> U, constexpr-param V>
     friend constexpr constant_wrapper<(U::value << V::value)> operator<<(U, V) { return {};
      }
  template <lhs-constexpr-param<type> U, constexpr-param V>
     friend constexpr constant_wrapper<(U::value >> V::value)> operator>>(U, V) { return {};
      }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value & V::value> operator&(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value | V::value> operator|(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value ^ V::value> operator^(U, V) { return {}; }

  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value && V::value> operator&&(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<U::value || V::value> operator||(U, V) { return {}; }

  template <lhs-constexpr-param<type> U, constexpr-param V>
     friend constexpr constant_wrapper<(U::value <=> V::value)> operator<=>(U, V) { return
      {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
     friend constexpr constant_wrapper<(U::value == V::value)> operator==(U, V) { return {};
      }
  template <lhs-constexpr-param<type> U, constexpr-param V>
```

```
      friend constexpr constant_wrapper<(U::value != V::value)> operator!=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value < V::value)> operator<(U, V) { return {}; }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value > V::value)> operator>(U, V) { return {}; }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value <= V::value)> operator<=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value >= V::value)> operator>=(U, V) { return {};
        }

    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value, V::value)> operator,(U, V) { return {}; }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value ->* V::value)> operator->*(U, V) { return
        {}; }

    template <auto Y = X>
      constexpr constant_wrapper<++Y> operator++() { return {}; }
    template <auto Y = X>
      constexpr constant_wrapper<Y++> operator++(int) { return {}; }
    template <auto Y = X>
      constexpr constant_wrapper<--Y> operator--() { return {}; }
    template <auto Y = X>
      constexpr constant_wrapper<Y--> operator--(int) { return {}; }

    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value += V::value)> operator+=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value -= V::value)> operator-=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value *= V::value)> operator*=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value /= V::value)> operator/=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value %= V::value)> operator%=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value &= V::value)> operator&=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value |= V::value)> operator|=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value ^= V::value)> operator^=(U, V) { return {};
        }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value <<= V::value)> operator<<=(U, V) { return
        {}; }
    template <lhs-constexpr-param<type> U, constexpr-param V>
      friend constexpr constant_wrapper<(U::value >>= V::value)> operator>>=(U, V) { return
        {}; }
  };

  template<auto X>
    inline constexpr constant_wrapper<X> cw{};
}
```

§ 10.2 Add a feature macro

Add a new feature macro, `__cpp_lib_constant_wrapper`.

§ 11 Implementation experience

Look up a few lines to see an implementation of `std::constant_wrapper`. At the time of this writing, there is one caveat: `operator[]()` looks correct to the authors, but does not work in any compiler tested, due to the very limited multi-variate `operator[]` support in even the latest compilers.

Additionally, an `integral_constant` with most of the operator overloads has been a part of Boost.Hana since its initial release in May of 2016. Its operations have been used by many, many users.

§ 12 Wording

Add the following to [meta.type.synop], after `false_type`:

```
template<auto X,
         class adl-type = remove_cvref_t<decltype(X)>>     // exposition only
  struct constant_wrapper;

template <class T>
  concept constexpr-param =                                // exposition only
    requires { typename constant_wrapper<T::value>; };
template <class T>
  concept derived-from-constexpr =                         // exposition only
    derived_from<T, constant_wrapper<T::value>>;
template <class T, class SelfT>
  concept lhs-constexpr-param =                            // exposition only
    constexpr-param<T> && (derived_from<T, SelfT> || !derived-from-constexpr<T>);

template<auto X>
  inline constexpr constant_wrapper<X> cw;
```

Add the following to [meta.help], after `integral_constant`:

```
template<auto X, class adl-type>
struct constant_wrapper {
  using value_type = adl-type;
  using type = constant_wrapper;

  constexpr operator value_type() const { return X; }
  static constexpr value_type value = X;

  template <constexpr-param U>
    constexpr constant_wrapper<(X = U::value)> operator=(U) const { return {}; }

  template<auto Y = X>
    constexpr constant_wrapper<+Y> operator+() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<-Y> operator-() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<~Y> operator~() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<!Y> operator!() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<&Y> operator&() const { return {}; }
  template<auto Y = X>
    constexpr constant_wrapper<*Y> operator*() const { return {}; }

  template<auto Y = X, class... Args>
    constexpr constant_wrapper<Y(Args::value...)> operator()(Args... args) const { return {};
        }
  template<auto Y = X, class... Args>
    constexpr constant_wrapper<Y[Args::value...]> operator[](Args... args) const { return {};
        }
  constexpr value_type operator()() const requires (!std::invocable<value_type>) { return X; }
```

```
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value + V::value> operator+(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value - V::value> operator-(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value * V::value> operator*(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value / V::value> operator/(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value % V::value> operator%(U, V) { return {}; }

template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value << V::value)> operator<<(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value >> V::value)> operator>>(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value & V::value> operator&(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value | V::value> operator|(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value ^ V::value> operator^(U, V) { return {}; }

template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value && V::value> operator&&(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<U::value || V::value> operator||(U, V) { return {}; }

template <lhs-constexpr-param<type> U, constexpr-param V>
   friend constexpr constant_wrapper<(U::value <=> V::value)> operator<=>(U, V) { return {};
      }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value == V::value)> operator==(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value != V::value)> operator!=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value < V::value)> operator<(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value > V::value)> operator>(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value <= V::value)> operator<=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value >= V::value)> operator>=(U, V) { return {}; }

template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value, V::value)> operator,(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
   friend constexpr constant_wrapper<(U::value ->* V::value)> operator->*(U, V) { return {};
      }

template <auto Y = X>
  constexpr constant_wrapper<++Y> operator++() { return {}; }
template <auto Y = X>
  constexpr constant_wrapper<Y++> operator++(int) { return {}; }
template <auto Y = X>
  constexpr constant_wrapper<--Y> operator--() { return {}; }
template <auto Y = X>
  constexpr constant_wrapper<Y--> operator--(int) { return {}; }

template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value += V::value)> operator+=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value -= V::value)> operator-=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value *= V::value)> operator*=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
  friend constexpr constant_wrapper<(U::value /= V::value)> operator/=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
```

```
    friend constexpr constant_wrapper<(U::value %= V::value)> operator%=(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<(U::value &= V::value)> operator&=(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<(U::value |= V::value)> operator|=(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<(U::value ^= V::value)> operator^=(U, V) { return {}; }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<(U::value <<= V::value)> operator<<=(U, V) { return {};
        }
  template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constant_wrapper<(U::value >>= V::value)> operator>>=(U, V) { return {};
        }
};

template<auto X>
  inline constexpr constant_wrapper<X> cw{};
```

2 The class template `constant_wrapper` aids in metaprogramming by ensuring that the evaluation of expressions comprised entirely of `constant_wrappers` are core constant expressions ([expr.const]), regardless of the context in which they appear. In particular, this enables use of `constant_wrapper` values that are passed as arguments to `constexpr` functions to be used as template parameters.

3 The variable template `cw` is provided as a convenient way to nominate `constant_wrapper` values.

Add to [version.syn]:

```
#define __cpp_lib_constant_wrapper XXXXXXL // also in <type_traits>
```